

## Agenda

1. What is Integrative programming
2. Why is it important?
3. What are the challenges?
4. What are the techniques for integration?

## Introduction

- Most applications in organizations are **heterogeneous, standalone** and perform **different functions**
- The main purpose of Integrative programming is to **integrate the heterogeneous** organizational resources to work together to perform the various functions.
- For example, an organization may have **data, programs** and **web resources** that need to be integrated (e.g., to use the programs to display data in the web pages)
- Application integration combines **heterogeneous, standalone** applications into one that seamlessly performs the various functions of the **standalone** applications
- Integrated programming is the process to achieve **application integration**
- In Integrative programming, **at least one component must exist at the time of integration**, e.g.,
  - There is a database application which provides reports
  - What we want (i.e., purpose of integration) is a new web interface to display the reports
  - **Question: any other example**
- There are various technologies and techniques for integration

## Reasons for integration: why integration

1. Heterogeneous environment: different platforms (data, software, hardware, etc.) which need to work together
2. Changes in technology which requires changes to existing components to work together
3. Changes in business processes, which require a new component to be integrated to existing application
4. “Distributed” organizational resources, e.g., data, applications, services, etc., at different locations which need to be integrated
5. Reduce latency (delays), transparency (“hidden” complexity)
6. High fault tolerance

## Different platform

1. OS, e.g., Windows, Linux, Mobile (iOS and Android)
2. Type of application, e.g., Web; Mobile
3. Programming language: Java, .Net, etc.
4. Data (from various sources: files, databases, spreadsheets, XML, etc.)

5. Scripting (JSP, ASP, PHP, etc.)
6. Other software (e.g., from different vendors)
7. Hardware (e.g., from different vendors)
8. Middleware

## Some important features of integration

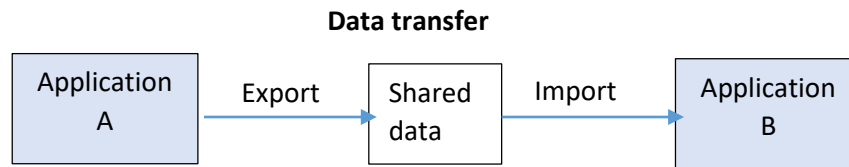
1. Data from various sources (e.g., files, databases) and different formats (e.g., spreadsheets, XML, etc.)
2. Applications on different platform
  - a. Programming languages: Java, .Net, etc.
  - b. Operating system: Web (Windows, Linux); Mobile (iOS and Android)
  - c. Scripting (JSP, ASP, PHP, etc.)
    - i. Middleware
    - ii. Legacy applications
    - iii. Vendor dependent software (e.g., from different vendors)
    - iv. Hardware (e.g., from different vendors)
3. Distributed applications
4. Networks
5. Message passing
6. Data exchange
7. Interoperability: protocols (e.g., communication, web service, etc.), data formats,
8. Security
9. Concurrency
10. Management
1. Architectures (distributed applications)
11. Technologies (for integration)
12. Etc.

## Challenges

1. Complexity: difficult to integrate heterogeneous platforms hardware/software/programming languages
2. Complexity: changes to one component may affect others components
3. Distribution and communication issues
  - i. Networks
  - ii. Communication protocols (CORBA, RMI, etc.)
  - iii. Data exchange
  - iv. Message passing (Synchronous and Asynchronous)
  - v. Multithreading
4. Security
5. Technologies for integration: which to select?

## Techniques

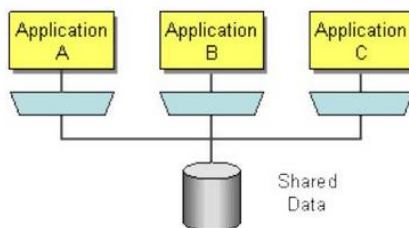
1. **File transfer:** one application writes data to a file; another reads from the file. The file can be binary, text, XML, Excel, etc.



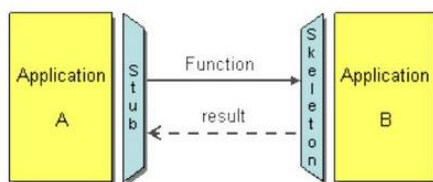
### Question

Assume A writes in Excel and B wants to read in text? What is the problem that can arise? How can this problem be avoided?

### 2. Shared database: applications share data base



### 3. Remote Procedure Call (Remote Method Invocation): Application “exposes” procedure (method). This is called (invoked) by another a remote. This can occur asynchronous or synchronous). There are several other ways for remote processes to communicate



**Figure 3 Remote procedure calls**

4. Middleware
5. Services
6. APIs

## Message passing

1. Widely used because of availability of networks
2. Synchronous v Asynchronous
3. Characteristics
  - i. Create: the sender creates the message (with necessary data)
  - ii. Send: the sender adds a message to a channel
  - iii. Deliver: the message system moves message from the sender to receiver (i.e., sending device to the receiver device)
  - iv. Receive: the receiver reads the message from the channel
  - v. Process: the receiver extracts the data from the message and performs the request
4. Note: if the receiver sends a response, then it becomes a sender and the steps a-e begins again.

**NOTE: we will discuss the concepts above in detail in this course**